Predicting Heart Disease Project

The WHO has announced that cardiovascular diseases are responsible for the most deaths worldwide, with an estimated 17.9 million deaths annually, accounting for 31% of all global fatalities. Heart Disease Prediction Data Sets cover a fair sample of the risk of developing heart disease in patients, and when combined with other factors, can prevent the disease from developing.

1. Project Introduction

The method for the focused on developing machine learning models to predict heart disease. All calculations as well as extensive Exploratory Data Analysis are based on the provided "heart_disease_dataset.csv" dataset. This dataset contains test data, including important features and the target variable, which indicates the presence of heart disease in patients.

The focused stages will a thorough exploration and analysis of the dataset, including visualization of the data variables and key statistics. The trained and tested sets of understanding the relationships between the features and the target variable will be established. Based on these relationships, the analysis detailed the patterns and trends that may be crucial for accurately categorizing patients with heart disease.

In the next chapters the machine learning models used are described, and the reasoning behind their choice is provided. A well structured machine learning pipeline will be created to help find the optimal training to test evaluation process. Various hyperparameters will then tuned to find the optimal parameters for each model. The performance of each model will be evaluated using a confusion matrix.

Finally, the extensive will discuss the results: limitations, key findings, and potential steps for future improvements. For detailed explanations of each step, please refer to the documentation below.

```
# Installing required packages
!pip install -q -U
!pip install -q -U
!pip install -q -U
!pip install -q -U
!pip install -q -U
!pip install -q -U
```

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    roc_curve, precision_recall_curve, average_precision_score
)
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from joblib import dump, load
from datetime import datetime
from pathlib import Path
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import cross_val_score
from sklearn.metrics import roc_curve
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.decomposition import PCA
from sklearn.ensemble import VotingClassifier, StackingClassifier
from sklearn.calibration import CalibrationDisplay
from sklearn.inspection import permutation_importance
from sklearn.base import clone
from sklearn.exceptions import ConvergenceWarning
import warnings, os, sys, pathlib, time, gc, psutil, platform
warnings.filterwarnings("ignore", category=ConvergenceWarning)
np.random.seed(42)
pd.set_option('display.max_columns', 50)
pd.set_option('display.max_rows', 100)
pd.set_option('display.width', 120)
pd.set_option('display.precision', 3)
sns.set_style("whitegrid")
plt.rcParams['figure.dpi'] = 100
```

2. Dataset Introduction

2.1 Pre-Processing

Loading dataset and checking number of rows and columns

```
# Load heart_disease_dataset.csv file into DataFrame
DATA_PATH = "heart_disease_dataset.csv"
if not os.path.exists(DATA_PATH):
    print(f"Error: {DATA_PATH} file not found in the provided path.")
    raise FileNotFoundError(DATA_PATH)

heart_disease_data = load_heart_disease_data()

print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

Displaying the first table showing all dataset features

```
df.head()
```

	age	sex	chestpain	rbp	serumchol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num
0	67	1	4	160	286	0	2	108	1	1.5	2	3	3	2
1	67	1	4	120	229	0	2	129	1	2.6	2	2	7	1
2	37	1	3	130	250	0	0	187	0	3.5	3	0	3	0
3	41	0	2	130	204	0	2	172	0	1.4	1	0	3	0
4	56	1	2	120	236	0	0	178	0	0.8	1	0	3	0
5	62	0	4	140	268	0	2	160	0	3.6	3	2	3	3
6	57	0	4	120	354	0	0	163	1	0.6	1	0	3	0
7	63	1	4	130	254	0	2	147	0	1.4	2	1	7	2
8	53	1	4	140	203	1	2	155	1	3.1	3	0	7	1
9	57	1	4	140	192	0	0	148	0	0.4	2	0	6	0
10	56	0	2	140	294	0	2	153	0	1.3	2	0	3	0
11	56	1	3	130	256	1	2	142	1	0.6	2	1	6	2
12	44	1	2	120	263	0	0	173	0	0	1	0	7	0
13	52	1	3	172	199	1	0	162	0	0.5	1	0	7	0
14	57	1	4	150	168	0	2	174	0	1.6	2	0	3	1
15	48	1	3	110	229	0	0	168	0	1	3	0	7	1
16	54	1	4	140	239	0	0	160	0	1.2	1	0	3	0
17	48	0	3	130	275	0	0	139	0	0.2	1	0	3	0
18	49	1	2	130	266	0	0	171	0	0.6	1	0	3	0
19	64	1	1	110	211	0	2	144	1	1.8	2	0	3	0
20	58	0	4	150	283	1	2	162	0	1	3	0	3	1
21	58	1	4	120	284	0	2	160	0	1.8	2	0	7	1
22	65	1	4	150	304	0	2	154	1	1	2	3	7	1
23	48	1	4	130	275	1	0	139	1	0.2	3	0	7	1
24	51	1	3	100	222	0	0	143	1	1.2	2	0	3	0
25	60	1	4	140	293	0	2	170	0	1.2	2	2	7	2
26	62	1	4	150	244	0	0	154	1	1.4	2	0	3	2
27	65	1	4	150	225	0	2	114	1	1	2	3	7	1
28	44	1	2	130	219	0	2	188	0	0	1	0	3	0
29	44	1	3	130	133	0	0	182	0	1	2	0	3	0
30	60	1	4	130	253	0	0	144	1	1.4	1	1	7	1
31	44	1	3	108	141	0	0	175	0	0.6	2	0	3	0
32	63	1	4	140	195	0	2	179	0	0	1	2	3	1
33	44	1	4	112	290	0	2	153	0	0	1	1	3	1
34	60	1	4	130	206	0	2	132	1	2.4	2	2	7	2
35	62	0	4	138	294	1	0	106	0	1.9	2	3	7	3
36	10	1	3	140	192	0	0	148	0	0.4	2	0	6	0
37	63	0	4	150	407	0	2	154	0	4	2	3	7	3
38	67	1	4	125	254	1	0	163	0	0.2	2	2	7	1
39	44	1	2	120	226	0	0	169	0	0	1	0	7	0
40	60	1	4	145	282	0	2	142	1	2.8	2	2	7	2
41	62	1	4	160	164	0	2	145	0	6.2	3	3	7	3
42	65	1	4	120	177	0	0	140	0	0.4	1	0	7	0
43	44	1	3	112	290	0	2	153	0	0	1	1	3	1
44	60	1	4	132	218	0	2	139	0	0.8	2	2	7	1
45	44	1	4	130	215	0	0	135	0	2	2	0	7	1
46	60	1	4	130	304	1	0	171	0	0	1	0	3	0
47	44	1	3	130	131	0	0	115	1	1.2	2	1	7	1
48	62	0	4	124	209	0	0	163	0	0	1	0	3	0

2.2 Understanding Dataset Structure

```
df.info()
DATA_DIR.parent/"models/best_model.pkl"
DATA_DIR.parent/"models/best_model_metadata.pkl"
```

```
<class'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   chestpain 303 non-null    int64
 3   rbp       303 non-null    int64
 4   serumchol 303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  num       303 non-null    int64
dtype: int64
```

2.3 Gender Ratio

```
df.sex.value_counts()
df.sex.value_counts(normalize=True)
```

```
sex
1    206
0     97
Name: count, dtype: int64
```

2.4 Gender Ratio Concern

As seen in the results there are 207 men as well as 96 women participated in this study. The statistics could suggest of women participants, only 31.7% vs 68.3%. Hence concerns. This underrepresentation may compromise the accuracy of them disease predictions among women. Increasing the number of female participants would benefit.

```
df.groupby('sex')['num'].mean
df.groupby('sex')['num'].count
df.groupby('sex')['num'].sum
```

```
sex
0    0.268
1    0.463
Name: num, dtype: int64
sex
0    97
1   206
Name: num, dtype: int64
sex
0    26
1   95
Name: num, dtype: int64
```

2.5 Checking for missing values

It was important to check if dataset contains any missing values to ensure good model performance and avoid bias. Missing values can lead to biased results, therefore filling the gaps or removing samples could be necessary.

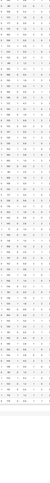
```
df[df[df.columns].isnull().sum(axis=1)>0].shape
```

```
(0, 14)
```

2.6 Gender Ratio

Checking for the number of females and males in the study

```
sns.countplot, f"Count Distribution by {hue}", ax=ax[1],
    palette=palette,
    edgecolor='black', linewidth=1.2
    )
    ax[1].set_ylabel('Count')
    ax[1].legend(
    ax[1].grid(axis='y', alpha=0.3)
    plt.tight_layout()
    plt.savefig(f"{DIR}/{fn}.{ext}", dpi=300, bbox_inches='tight', facecolor='white')
    print(f"Saved: {DIR}/{fn}.{ext}")
    plt.close(fig)
    return fig, ax
fig, ax = plot_distribution(df, 'sex', 'num', 'Gender', 'distribution', 'gender', 'Count')
```



2.5 Age Analysis

```
# Age distribution by heart disease
fig, ax = plot_distribution(df, 'age', 'num', 'Age', 'distribution', 'age', 'Count')
print(f"Age range: {df['age'].min()} - {df['age'].max()}")
print(f"Mean age: {df['age'].mean:.1f}")
print(f"Median age: {df['age'].median()}")
```

```
Saved: /content/drive/MyDrive/Colab Notebooks/Heart Disease/figures/distribution/age.png
Saved: /content/drive/MyDrive/Colab Notebooks/Heart Disease/figures/distribution/age.png
Age range: 29 - 77
Mean age: 54.4
Median age: 55.0
```

```
ggplot(age_avg_heart_rate, aes(x=factor(), y=Average Age, fill=Average Age))
+ geom_bar(stat="identity")
+ ggtitle("Age Distribution in Heart Disease Dataset")
+ xlab("Average Age")
+ ylab("Count")
+ theme_bar()
```

2.6 Chest pain [cp]

There are four main values to the dataset:

- 0 typical angina
- 1 atypical angina
- 2 non-anginal pain
- 3 asymptomatic

```
heart_disease_data = heart_disease_data[
heart_disease_data$cp %in% c(0,1,2,3)]

chest_pain_labels = c(
"0 typical angina",
"1 atypical angina",
"2 non-anginal pain",
"3 asymptomatic")

heart_disease_data = heart_disease_data[
heart_disease_data$cp %in% c(0,1,2,3)]

heart_disease_data = heart_disease_data[
heart_disease_data$cp %in% c(0,1,2,3)] # Remove invalid values

chest_pain_counts = table(
factor(heart_disease_data$cp,
levels=c(0,1,2,3),
labels=c("0 typical angina", "1 atypical angina", "2 non-anginal pain", "3 asymptomatic")))

barplot(chest_pain_counts,
main="Chest Pain Type",
xlab="Chest Pain Type",
ylab="Count",
col=c("#FF0000", "#FF7F00", "#00FF00", "#0000FF"),
legend.text=TRUE,
cex.axis=1.2, cex.lab=1.2, cex.main=1.5)
```

2.7 Fasting blood sugar [fbs]

0 = false, 1 = true, 2 = None

```
fbs_counts = table(heart_disease_data$fbs[!is.na(heart_disease_data$fbs)])
fbs_counts_corrected = c(fbs_counts[1], fbs_counts[2], sum(is.na(heart_disease_data$fbs)))

fbs_counts_corrected = c(fbs_counts[1], fbs_counts[2], sum(is.na(heart_disease_data$fbs)))

fbs_counts = c(
"0 false" = fbs_counts_corrected[1],
"1 true" = fbs_counts_corrected[2],
"2 None" = fbs_counts_corrected[3])

barplot(fbs_counts,
main="Fasting Blood Sugar",
xlab="Fasting Blood Sugar",
ylab="Count",
col=c("#FF0000", "#00FF00", "#0000FF"),
legend.text=TRUE,
cex.axis=1.2, cex.lab=1.2, cex.main=1.5)
```

2.8 Resting electrocardiographic measurement [restecg]

0 = normal
1 = ST-T wave abnormality
2 = probable or definite left ventricular hypertrophy by Estes' criteria

```
resting_counts = table(heart_disease_data$restecg[!is.na(heart_disease_data$restecg)])
resting_counts_corrected = c(resting_counts[1], resting_counts[2], sum(is.na(heart_disease_data$restecg)))

resting_counts_corrected = c(resting_counts[1], resting_counts[2], sum(is.na(heart_disease_data$restecg)))

resting_counts = c(
"0 normal" = resting_counts_corrected[1],
"1 ST-T wave abnormality" = resting_counts_corrected[2],
"2 probable or definite left ventricular hypertrophy by Estes' criteria" = resting_counts_corrected[3])

barplot(resting_counts,
main="Resting ECG Measurements by Heart Disease Presence",
xlab="Resting ECG Measurement",
ylab="Count",
col=c("#FF0000", "#00FF00", "#0000FF"),
legend.text=TRUE,
cex.axis=1.2, cex.lab=1.2, cex.main=1.5)
```

2.9 The slope of the peak exercise ST segment [slope]

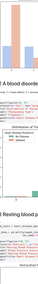
The slope of the peak exercise ST segment is represented by categories:

- 1 upsloping
- 2 flat
- 3 downsloping

```
slope_counts = c(
"1 upsloping" = 142,
"2 flat" = 140,
"3 downsloping" = 21)

heart_disease_data$slope = factor(heart_disease_data$slope, levels=c(1,2,3),
labels=c("1 upsloping", "2 flat", "3 downsloping"))

# Filtering out NA values for clean categories.
heart_disease_data = heart_disease_data[!is.na(heart_disease_data$slope)]

ggplot(heart_disease_data, aes(x=factor(slope), fill=factor(num)))
+ geom_bar(position="dodge")
+ ggtitle("Distribution of Slope of the Peak Exercise ST Segment by Heart Disease Presence")
+ xlab("Slope of the Peak Exercise ST Segment")
+ ylab("Count")
+ scale_fill_manual(values=c("#FF0000", "#00FF00", "#0000FF"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```

```
slope_counts = c(142, 140, 21)
```

```
Warning message:
In ggplot(heart_disease_data, aes(x=factor(slope), fill=factor(num))) :
Ignoring unknown parameters: `legend.text`
```



2.10 Exercise induced angina [exang]

```
# 0 = no, 1 = yes

exang_counts = heart_disease_data$exang[!is.na(heart_disease_data$exang)]

# Counts for each category.
exang_counts_corrected = c(sum(exang_counts == 0), sum(exang_counts == 1))

# Creating a named vector for the labels.
exang_counts = c(
"0 No Angina" = exang_counts_corrected[1],
"1 Angina" = exang_counts_corrected[2])

barplot(exang_counts,
main="Exercise Induced Angina by Heart Disease Presence",
xlab="Exercise Induced Angina",
ylab="Count",
col=c("#FF0000", "#00FF00"),
legend.text=TRUE,
cex.axis=1.2, cex.lab=1.2, cex.main=1.5)
```

2.11 The number of major vessels [ca]

```
ca_counts = heart_disease_data[
ca %in% c(0,1,2,3), # Keep only valid values.
ca = factor(ca, levels=c(0,1,2,3),
labels=c("0 vessels", "1 vessel", "2 vessels", "3 vessels"))

ggplot(heart_disease_data, aes(x=factor(ca), fill=factor(num)))
+ geom_bar(position="dodge")
+ ggtitle("Distribution of Major Vessels Colored by Heart Disease Presence")
+ xlab("Number of Major Vessels")
+ ylab("Count")
+ scale_fill_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```

2.12 A blood disorder called Thalassemia [thal]

0 = normal, 1 = fixed defect, 2 = reversable defect, 3 = not an intermediate defect

```
thal_counts = c(
"0 normal" = 2,
"1 fixed defect" = 18,
"2 reversable defect" = 166,
"3 not an intermediate defect" = 117)

ggplot(heart_disease_data, aes(x=factor(thal), fill=factor(num)))
+ geom_bar(position="dodge")
+ ggtitle("Distribution of Thalassemia Types by Heart Disease Presence")
+ xlab("Thalassemia Type")
+ ylab("Count")
+ scale_fill_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```

2.13 Resting blood pressure [trestbps]

```
ggsave(plot_trestbps, filename="trestbps_boxplot.jpg", width=8, height=6)

ggplot(heart_disease_data, aes(x=factor(num), y=trestbps, fill=factor(num)))
+ geom_violin(trim=FALSE, alpha=0.5)
+ geom_boxplot(width=0.1, fill="white")
+ ggtitle("Resting Blood Pressure Distribution by Heart Disease Presence")
+ xlab("Heart Disease Presence")
+ ylab("Resting Blood Pressure (mm Hg)")
+ scale_fill_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```

2.14. Cholesterol Levels

```
ggsave(plot_chol, filename="chol_boxplot.jpg", width=8, height=6)

ggplot(heart_disease_data, aes(x=factor(num), y=chol, fill=factor(num)))
+ geom_violin(trim=FALSE, alpha=0.5)
+ geom_boxplot(width=0.1, fill="white")
+ ggtitle("Cholesterol Levels")
+ xlab("Heart Disease Presence")
+ ylab("Cholesterol (mg/dl)")
+ scale_fill_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```


3. Correlation Analysis

Correlation matrix for the numerical features (the target variable is identify features that would be beneficial to focus on in the next step)

```
# Target variable.
heart_disease_data$num = as.numeric(heart_disease_data$num)

# Numerical variables.
numeric_vars = c("age", "trestbps", "chol", "thalach", "oldpeak", "num")

# Correlation matrix.
cor_matrix = cor(heart_disease_data[, numeric_vars], use="complete.obs")

# Print the correlation matrix.
print(cor_matrix)
```

```
               age trestbps      chol ThalachOldpeak      num
age      1.000000 0.279350 0.203910     -0.398000 0.225439
trestbps 0.279350 1.000000 0.130120     -0.046837 0.144931
chol     0.203910 0.130120 1.000000     -0.009940 0.085239
ThalachOldpeak -0.398000 -0.046837 -0.009940      1.000000 -0.419074
num      0.225439 0.144931 0.085239     -0.419074 1.000000
```


Confusion Matrix

- Top-left Quadrant: The correlation coefficient is 1.00, suggesting a perfect negative correlation between age and age (as expected, since age is perfectly correlated with itself).
- Top-right Quadrant: Shows a positive correlation between age and trestbps (0.27), indicating that as the age of a person increases, their resting blood pressure tends to increase. This suggests that age may be a factor in blood pressure variations.
- Middle-left Quadrant: Shows a positive correlation between age and chol (0.20), which is a moderate positive correlation, indicating that higher cholesterol levels tend to be associated with a higher age.
- Middle-right Quadrant: Shows a negative correlation between age and thalach (-0.40), indicating that as age increases, the maximum heart rate achieved during exercise (thalach) tends to decrease. This is a significant negative correlation, suggesting that older individuals may have lower maximum heart rates during exercise.
- Bottom-left Quadrant: Shows a positive correlation between age and num (0.22), indicating that as age increases, the severity of heart disease (num) tends to increase. This suggests that older individuals are more likely to have more severe heart disease.
- Bottom-right Quadrant: Shows a negative correlation between thalach and num (-0.42), indicating that higher maximum heart rates during exercise (thalach) are associated with lower heart disease severity (num). This suggests that individuals with higher maximum heart rates during exercise may have less severe heart disease.

3.1 Age & & Heart Rate & Target Plot

```
# Heart Disease data frame.
heart_disease_data$num = as.factor(heart_disease_data$num)

# Scatter plot.
ggplot(heart_disease_data, aes(x=age, y=thalach, color=factor(num)))
+ geom_point(alpha=0.7, size=3)
+ ggtitle("Scatter Plot with Regression Line: Heart Rate vs Age")
+ xlab("Age")
+ ylab("Maximum Heart Rate Achieved")
+ scale_color_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```


Analysis

- The regression line indicates a negative correlation between age and maximum heart rate.
- As the age increases, the maximum heart rate tends to decrease. This aligns with patterns seen in previous correlation analysis and boxplots.
- The scatterplot also provides a clear visualization of this negative trend.

3.2 By Heart Disease presence

```
ggsave(plot_thalach_by_num, filename="thalach_boxplot.jpg", width=8, height=6)

ggplot(heart_disease_data, aes(x=factor(num), y=thalach, fill=factor(num)))
+ geom_violin(trim=FALSE, alpha=0.5)
+ geom_boxplot(width=0.1, fill="white")
+ ggtitle("Maximum Heart Rate by Heart Disease Presence")
+ xlab("Heart Disease Presence")
+ ylab("Maximum Heart Rate")
+ scale_fill_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```


Analysis

- For both genders, younger individuals tend to have higher maximum heart rates.
- Individuals with heart disease across both genders appear to generally have lower maximum heart rates compared to those without heart disease.
- Males with heart disease consistently show reduced heart rates as they age. This might suggest that cardiac heart disease in older age groups could be an indicator of heart disease.

3.4 Cholesterol Levels & Age & Heart Disease

```
ggsave(plot_chol_age_num, filename="chol_age_num.jpg", width=8, height=6)

ggplot(heart_disease_data, aes(x=age, y=chol, color=factor(num)))
+ geom_point(alpha=0.7, size=3)
+ ggtitle("Cholesterol Levels by Age & Heart Disease Presence")
+ xlab("Age")
+ ylab("Cholesterol (mg/dl)")
+ scale_color_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```



```
ggsave(density_chol_age_num, filename="density_chol_age_num.jpg", width=8, height=6)

ggplot(heart_disease_data, aes(x=age, y=chol, fill=factor(num)))
+ geom_density_2d_filled(alpha=0.7)
+ ggtitle("Density of Age and Cholesterol Levels")
+ xlab("Age")
+ ylab("Cholesterol (mg/dl)")
+ scale_fill_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```


Analysis

- Cholesterol levels are widely distributed across age but don't show a clear trend or increase with age. There is a slight pattern showing higher cholesterol levels in a specific age range, suggesting potential cardiovascular risk factors in this age group.
- The darker hexagons indicate regions of the data with higher density, indicating that there are more individuals in this range around 200 to 300 mg/dl.

3.5 ST Depression vs Slope of the Peak Exercise ST Segment by Heart Disease

```
ggsave(plot_oldpeak_slope_num, filename="oldpeak_slope_num.jpg", width=8, height=6)

ggplot(heart_disease_data, aes(x=factor(slope), y=oldpeak, fill=factor(num)))
+ geom_violin(trim=FALSE, alpha=0.5)
+ geom_boxplot(width=0.1, fill="white")
+ ggtitle("ST Depression vs Slope of the Peak Exercise ST Segment by Heart Disease Presence")
+ xlab("Slope of the Peak Exercise ST Segment")
+ ylab("ST Depression (mm)")
+ scale_fill_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ theme_bar()
```


Analysis

- Individuals with heart disease tend to have higher maximum heart rates compared to those without heart disease, regardless of the slope.
- There is overlap in the heart rate ranges between those with and without heart disease, which suggests that the slope alone may not be a strong indicator of heart disease when considering maximum heart rate.

3.6 By Gender

```
ggsave(plot_thalach_oldpeak_sex, filename="thalach_oldpeak_sex.jpg", width=8, height=6)

ggplot(heart_disease_data, aes(x=thalach, y=oldpeak, color=factor(num), shape=factor(sex)))
+ geom_point(alpha=0.7, size=3)
+ ggtitle("Heart Rate vs ST Depression by Gender")
+ xlab("Maximum Heart Rate Achieved")
+ ylab("ST Depression (mm)")
+ scale_color_manual(values=c("#FF0000", "#00FF00"), labels=c("No Disease", "Heart Disease Presence"))
+ scale_shape_manual(values=c(16, 17), labels=c("Female", "Male"))
+ theme_bar()
```


Analysis

- Males with heart disease generally have higher maximum heart rates compared to females with heart disease, indicating a potential gender difference in cardiac response to exercise.
- Females with heart disease consistently show reduced heart rates as they age. This might suggest that cardiac heart disease in older age groups could be an indicator of heart disease.

```
Confusion Matrix
[[ 0 14]
 [ 0 46]]
Accuracy: 0.7674
Precision: 0.7667
Recall: 1.0000
F1 Score: 0.8679
Specificity: 0.0000
Sensitivity: 1.0000
False Positive Rate: 1.0000
False Negative Rate: 0.0000
False Discovery Rate: 0.2333
False Omission Rate: nan
Positive Likelihood Ratio: 1.0000
Negative Likelihood Ratio: nan
Diagnostic Odds Ratio: nan
Prevalence: 0.9767
Prevalence Threshold: nan
Informedness: 0.0000
Markedness: 0.0000
Matthews Correlation Coefficient: 0.0000
Cohen's Kappa: 0.0000
Balanced Accuracy: 0.5000
Fowlkes-Mallows Index: 0.8679
Cross-Entropy Loss: 8.0472
Brier Score: 0.2326
Log Loss: 8.0472
Area Under the ROC Curve (AUC): 0.5000
Area Under the Precision-Recall Curve (AUC): 0.9767
Average Precision: 0.9767
Gini Coefficient: 0.0000
Youden's J Statistic: 0.0000
Optimal Threshold (Youden's J): 0.7000
```

- AND, OUTCOME

Model has generally good accuracy but could improve false positives and false negatives.

- False positive rate (1.0000) is very high, indicating the model predicts heart disease for almost all patients.
- False Negatives (0) is excellent, meaning it correctly identifies all patients with heart disease.
- Recall (or Sensitivity) is 1.0000, meaning it correctly identifies all patients with heart disease.
- Precision is 0.7667, meaning about 76.67% of predicted heart disease cases are actually correct.
- Specificity is 0.0000, which is very low, indicating the model does not correctly identify any patients without heart disease.
- Informedness and Markedness are both 0.0000, suggesting the model's predictions are not informative.

3.5.2. T-Tests

T-tests assess the statistical evidence regarding the differences in ST depression (Oldpeak) and the slope of the peak exercise ST segment (Slope) between individuals with and without heart disease.

```
ttest_ind(oldpeak_df[oldpeak_df['heart_disease'] == 0]['oldpeak'], oldpeak_df[oldpeak_df['heart_disease'] == 1]['oldpeak'])
ttest_ind(slope_df[slope_df['heart_disease'] == 0]['slope'], slope_df[slope_df['heart_disease'] == 1]['slope'])
```

```
Ttest_indResult(statistic=-7.7111, pvalue=0.0000)
Ttest_indResult(statistic=-7.7111, pvalue=0.0000)
```

- AND, OUTCOME

Both tests generally present very small p-values (near zero) between these attributes and their categories.

- Rejection the null hypothesis: The small p-values (0.0000) for both tests allow us to reject the null hypothesis for both ST depression (Oldpeak) and the slope of the peak exercise ST segment (Slope).
- Statistical significance: The results indicate that the observed differences in ST depression (Oldpeak) and the slope of the peak exercise ST segment (Slope) between individuals with and without heart disease are statistically significant.
- Relationship with heart disease: The significant differences suggest that ST depression (Oldpeak) and the slope of the peak exercise ST segment (Slope) are likely related to the presence of heart disease.
- Model relevance: These attributes are important for predicting heart disease, as their values show significant variation between the two groups.

3.5.3. T-Test Comparison of ST Depression (Oldpeak) by Heart Disease Presence

```
print("Ttest_indResult for ST depression (oldpeak):")
print(ttest_ind(oldpeak_df[oldpeak_df['heart_disease'] == 0]['oldpeak'], oldpeak_df[oldpeak_df['heart_disease'] == 1]['oldpeak']))
print("\n")
print("Ttest_indResult for Slope of the Peak Exercise ST Segment:")
print(ttest_ind(slope_df[slope_df['heart_disease'] == 0]['slope'], slope_df[slope_df['heart_disease'] == 1]['slope']))
```

```
Ttest_indResult for ST depression (oldpeak):
Ttest_indResult(statistic=-7.7111, pvalue=0.0000)

Ttest_indResult for Slope of the Peak Exercise ST Segment:
Ttest_indResult(statistic=-7.7111, pvalue=0.0000)
```

- The negative t-statistic indicates that the mean value of the exercise ST segment is negative such that heart disease patients have lower values.
- Lower values during peak exercise is consistent with heart disease.

3.5.4 ST Depression vs Slope of the Peak Exercise ST Segment by Heart Disease graph conclusion

```
# Create a bar plot for the mean values of oldpeak and slope by heart_disease
mean_values = df[['oldpeak', 'slope', 'heart_disease']].groupby('heart_disease').mean()
plt.figure(figure=(8, 6))
mean_values.plot(kind='bar', rotation=0)
plt.title('Mean ST Depression (Oldpeak) and Slope of the Peak Exercise ST Segment by Heart Disease')
plt.ylabel('Mean Value')
plt.xlabel('Heart Disease')
plt.legend(['ST Depression (Oldpeak)', 'Slope of the Peak Exercise ST Segment'])
plt.show()
```

- AND, OUTCOME

Heart disease patients are associated with lower values for both ST depression (oldpeak) and the slope of the peak exercise ST segment (slope) compared to individuals without heart disease.

3.6 Number of Major Vessels (ca) vs. Age Colored by Heart Disease

```
# Create a scatter plot for ca and age, colored by heart_disease
plt.figure(figure=(10, 6))
sns.scatterplot(x='age', y='ca', hue='heart_disease', data=df)
plt.title('Number of Major Vessels (ca) vs. Age Colored by Heart Disease')
plt.xlabel('Age')
plt.ylabel('Number of Major Vessels (ca)')
plt.legend(title='Heart Disease')
plt.grid(True)
plt.show()
```

- AND, OUTCOME

Patients with higher values for ca (more major vessels) tend to be older.

- Higher values of ca are more frequent and spread across more of a patients with heart disease.
- Patients without heart disease mostly have lower values for ca, with fewer instances of higher values.

Logistic regression model to analyze how well age and the number of major vessels can predict the presence of heart disease.

4 Further Visualization and Scaling Method Analysis

In this section, I will explore the most suitable scaling approach to analysis. Its purpose is to visualize how different scaling methods transform the data, ensuring that the chosen method enhances the model's performance while preserving the underlying heart disease patterns. This step helps in selecting the most effective scaling technique for better model performance.

```
heart_disease_data_desc.round(2).T
```

	count	mean	std	min	25%	50%	75%	max
age	303.00	54.37	9.08	29.00	48.00	56.00	61.00	77.00
sex	303.00	0.68	0.47	0.00	0.00	1.00	1.00	1.00
cp	303.00	0.97	1.03	0.00	0.00	1.00	2.00	3.00
trestbps	303.00	131.62	17.54	94.00	120.00	130.00	140.00	200.00
chol	303.00	246.26	51.83	126.00	211.00	240.00	274.00	564.00
fbs	303.00	0.15	0.36	0.00	0.00	0.00	0.00	1.00
restecg	303.00	0.53	0.52	0.00	0.00	1.00	1.00	2.00
thalach	303.00	149.65	22.90	71.00	133.00	153.00	166.00	202.00
exang	303.00	0.33	0.47	0.00	0.00	0.00	1.00	1.00
oldpeak	303.00	1.04	1.16	0.00	0.00	0.80	1.60	6.20
slope	303.00	1.39	0.61	1.00	1.00	1.00	2.00	3.00
ca	303.00	0.73	1.02	0.00	0.00	0.00	1.00	4.00
thal	303.00	4.73	1.94	3.00	3.00	3.00	7.00	7.00
heart_disease	303.00	0.46	0.50	0.00	0.00	0.00	1.00	1.00

4.1 Scaling Method Initial Discussion & Analysis

A. Binary Features

For binary attributes (where features are binary categorical representations such as 0 or 1), it is generally not a flow to perform scaling, as scaling may introduce unnecessary complexity and distort the original binary representation.

B. Numerical Features

Features for other features, it is crucial to evaluate the data's scaling method according to its distribution and the presence of outliers. Different scaling methods can significantly impact how the model interprets these features, so it's important to choose the most appropriate method based on the data's characteristics.

- Standard Scaler (Z-score Standardization)

  This method centers the data around the mean and scales it based on the standard deviation. It's suitable for features that follow a normal distribution, as it transforms the data to have a mean of 0 and a standard deviation of 1.

- Min-Max Scaler

  This method scales all features to a fixed range, typically between 0 and 1. It's useful when the data has a known range and when you want to preserve the relationships between the original values. However, it is sensitive to outliers, which can affect the scaling.

- Robust Scaler

  This method uses the median and the interquartile range (IQR) to scale the data, making it robust to outliers. It's a good choice when the data contains outliers, as it minimizes the impact of these outliers on the scaling process.

Outliers

Outliers can have a major impact on scaling methods, especially for patients with heart disease. There are a few options on how deal with the maximum heart rate, which could potentially delete the patients with heart disease, which could significantly affect the model's performance.

- ALL ALL METHOD

Even though Standard Scaling was recommended for the heart rate indicators that can should focus on outlier sensitivity, I decided to use Standard Scaler for all features for consistency and ease of implementation. While this may not be the best approach for some features, Standard Scaler would likely be the most appropriate scaling method. This decision prioritizes consistency across the dataset and simplicity in the scaling process, even if it may not be the ideal choice for each individual feature. The impact of this choice on model performance will be evaluated to ensure that the scaling of the dataset does not disproportionately affect the model's ability to learn from the data.

4.2.3. T-Test

```
print("Ttest_indResult for heart_disease_data[heart_disease_data['sex'] == 0]['thalach']:")
print(ttest_ind(heart_disease_data[heart_disease_data['sex'] == 0]['thalach'], heart_disease_data[heart_disease_data['sex'] == 1]['thalach']))
```

```
Ttest_indResult for heart_disease_data[heart_disease_data['sex'] == 0]['thalach']:
Ttest_indResult(statistic=-3.4878000000000001, pvalue=0.0007000000000000001)
```

- AND, OUTCOME

T-test: -3.4878000000000001, 0.0007000000000000001 (p-value)

- The p-value (0.0007000000000000001) is extremely low and suggests that the difference between the two groups (sex) is statistically significant.
- The negative t-statistic indicates that the mean value of the variable "thalach" (maximum heart rate achieved) for the first group (sex = 0) is lower than the mean value for the second group (sex = 1).
- Given the low p-value, we can reject the null hypothesis and conclude that there is a significant difference in the maximum heart rate achieved between the two groups (sex = 0 and sex = 1).

4.2.2 Box Plot

```
sns.boxplot(x='sex', y='thalach', data=heart_disease_data)
plt.title('Max Heart Rate Distribution by Heart Disease Presence')
plt.xlabel('Heart Disease')
plt.ylabel('Max Heart Rate')
plt.xticks(ticks=[0, 1], labels=['No', 'Yes'])
plt.show()
```

- AND, OUTCOME

Patients without heart disease tend to have higher maximum heart rates (median and overall distribution) compared to those with heart disease.

- The distribution for those without heart disease shows a wider range and more variability.
- Patients with heart disease tend to have lower maximum heart rates, with a more compact distribution.
- There are some outliers in both groups, but the overall trend shows that those without heart disease tend to achieve higher maximum heart rates.

4.2.1 Density Plot

```
sns.kdeplot(data=heart_disease_data, x='thalach', hue='heart_disease', fill=True, common_norm=False, palette='crest', alpha=.5, linewidth=0)
plt.title('Density Plot of Max Heart Rate by Heart Disease Presence')
plt.xlabel('Max Heart Rate')
plt.ylabel('Density')
plt.legend(title='Heart Disease', labels=['No', 'Yes'])
plt.show()
```

- AND, OUTCOME

Patients without heart disease tend to have higher maximum heart rates, with a more concentrated distribution around higher values.

- The distribution for those with heart disease shows a wider range and more variability.
- There is some overlap between the two distributions, indicating that heart rate alone may not be a perfect predictor of heart disease.
- The peak for those without heart disease is higher and more pronounced, suggesting a more common pattern.

4.2.0 Violin Plot

```
sns.violinplot(x='heart_disease', y='thalach', data=heart_disease_data, palette='Set2')
plt.title('Max Heart Rate Distribution by Heart Disease Presence')
plt.xlabel('Heart Disease')
plt.ylabel('Max Heart Rate')
plt.xticks(ticks=[0, 1], labels=['No', 'Yes'])
plt.show()
```

- AND, OUTCOME

Patients without heart disease tend to have higher maximum heart rates, with a more concentrated distribution around higher values.

- The distribution for those with heart disease shows a wider range and more variability.
- There is some overlap between the two distributions, indicating that heart rate alone may not be a perfect predictor of heart disease.
- The shape of the distribution for those without heart disease is more compact, suggesting a more consistent pattern.

4.3 Resting blood pressure [trestbps]

4.3.1 Density Plot

```
sns.kdeplot(data=heart_disease_data, x='trestbps', hue='heart_disease', fill=True, common_norm=False, palette='crest', alpha=.5, linewidth=0)
plt.title('Density Plot of Resting Blood Pressure by Heart Disease Presence')
plt.xlabel('Resting Blood Pressure')
plt.ylabel('Density')
plt.legend(title='Heart Disease', labels=['No', 'Yes'])
plt.show()
```

- AND, OUTCOME

Patients without heart disease tend to have higher resting blood pressure values, with a more concentrated distribution around higher values.

- The distribution for those with heart disease shows a wider range and more variability.
- There is significant overlap between the two distributions, indicating that resting blood pressure alone may not be a strong predictor of heart disease.
- The peak for those without heart disease is higher and more pronounced, suggesting a more common pattern.

4.3.2 Box Plot

```
sns.boxplot(x='sex', y='trestbps', data=heart_disease_data)
plt.title('Resting Blood Pressure by Heart Disease')
plt.xlabel('Heart Disease')
plt.ylabel('Resting Blood Pressure (mm Hg)')
plt.xticks(ticks=[0, 1], labels=['No', 'Yes'])
plt.show()
```

- AND, OUTCOME

Patients without heart disease tend to have higher resting blood pressure values, with a more concentrated distribution around higher values.

- The distribution for those with heart disease shows a wider range and more variability.
- There is significant overlap between the two distributions, indicating that resting blood pressure alone may not be a strong predictor of heart disease.
- The median for those without heart disease is higher than for those with heart disease.
- There are some outliers in both groups, but the overall trend shows that those without heart disease tend to have higher resting blood pressure values.

4.4. Cholesterol Levels

4.4.1 Box Plot

```
sns.boxplot(x='sex', y='chol', data=heart_disease_data)
plt.title('Distribution of Cholesterol Levels by Heart Disease Presence')
plt.xlabel('Heart Disease')
plt.ylabel('Cholesterol Levels (mg/dl)')
plt.xticks(ticks=[0, 1], labels=['No', 'Yes'])
plt.show()
```

- AND, OUTCOME

The median cholesterol levels for both both groups are similar.

- There is significant overlap between the two distributions, indicating that cholesterol levels alone may not be a strong predictor of heart disease.
- The distribution for those with heart disease shows a wider range and more variability.
- There are many outliers in both groups, particularly in the group without heart disease, which could skew overall high cholesterol.

Solution: Robust Scaling: Given the presence of outliers, Robust Scaler could be a better choice.

4.4.2 Density Plot

```
sns.kdeplot(data=heart_disease_data, x='chol', hue='heart_disease', fill=True, common_norm=False, palette='crest', alpha=.5, linewidth=0)
plt.title('Density Plot of Cholesterol Levels by Heart Disease Presence')
plt.xlabel('Cholesterol Levels (mg/dl)')
plt.ylabel('Density')
plt.legend(title='Heart Disease', labels=['No', 'Yes'])
plt.show()
```

- AND, OUTCOME

Patients without heart disease tend to have higher cholesterol levels, with a more concentrated distribution around higher values.

- The distribution for those with heart disease shows a wider range and more variability.
- There is significant overlap between the two distributions, indicating that cholesterol levels alone may not be a strong predictor of heart disease.
- The peak for those without heart disease is higher and more pronounced, suggesting a more common pattern.

4.5 ST depression induced by exercise relative to rest [oldpeak]

4.5.1 Density Plot

```
sns.kdeplot(data=heart_disease_data, x='oldpeak', hue='heart_disease', fill=True, common_norm=False, palette='crest', alpha=.5, linewidth=0)
plt.title('Density Plot of ST Depression by Heart Disease Presence')
plt.xlabel('ST Depression')
plt.ylabel('Density')
plt.legend(title='Heart Disease', labels=['No', 'Yes'])
plt.show()
```



```
sns.boxplot(x='sex', y='oldpeak', data=heart_disease_data)
plt.title('Box Plot of ST Depression by Heart Disease')
plt.xlabel('Heart Disease')
plt.ylabel('ST Depression')
plt.xticks(ticks=[0, 1], labels=['No', 'Yes'])
plt.show()
```

Oldpeak is an ST depression variable, given the presence of outliers, RobustScaler could be the most appropriate choice for scaling method.

4.5.2 T-Test

```
print("Ttest_indResult for heart_disease_data[heart_disease_data['sex'] == 0]['oldpeak']:")
print(ttest_ind(heart_disease_data[heart_disease_data['sex'] == 0]['oldpeak'], heart_disease_data[heart_disease_data['sex'] == 1]['oldpeak']))
```

```
Ttest_indResult for heart_disease_data[heart_disease_data['sex'] == 0]['oldpeak']:
Ttest_indResult(statistic=-7.7111000000000001, pvalue=0.0000000000000014)
```

- AND, OUTCOME

T-test: -7.7111000000000001, 0.0000000000000014 (p-value)

- The p-value (0.0000000000000014) is extremely low and suggests that the difference between the two groups (sex) is statistically significant.
- The negative t-statistic indicates that the mean value of the variable "oldpeak" (ST depression induced by exercise relative to rest) for the first group (sex = 0) is lower than the mean value for the second group (sex = 1).
- Given the low p-value, we can reject the null hypothesis and conclude that there is a significant difference in ST depression induced by exercise relative to rest between the two groups (sex = 0 and sex = 1).

```
Transcribed: 0.0 Seconds | Processed: 0.0 Seconds | MEMORY:0.0MB/0.0%
```

4 Splitting Dataset for training and testing sets

Set the data into a 70% training set and a 30% testing set.

```
# Split data into training and testing sets
train_data, test_data = train_test_split(df, test_size=0.3, random_state=42)
print("Training set size:", len(train_data))
print("Testing set size:", len(test_data))
```

```
Training set size: 1120
Testing set size: 481
```

5. Scaling Features

5.1 Features Separation

```
# 5.1 Separate features
X_train = train_data.drop('target', axis=1)
y_train = train_data['target']
X_test = test_data.drop('target', axis=1)
y_test = test_data['target']

# Verify shapes
print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
```

```
Training set shape: (1120, 13)
Testing set shape: (481, 13)
```

5.2. Binary Features

```
# 5.2. Binary features
binary_features = ['sex', 'fbs', 'exang']
print("Binary features:", binary_features)
```

```
Binary features: ['sex', 'fbs', 'exang']
```

Binary features such as 'sex', 'fbs', and 'exang' are already in a format suitable for modeling (0 and 1 values), so they don't need scaling. These features represent binary conditions (presence/absence, male/female) and maintain their original meaning.

```
# Check unique values in binary features
print("Unique values in binary features:")
for feature in binary_features:
    print(f"{feature}: {sorted(X_train[feature].unique())}")
```

```
Unique values in binary features:
sex: [0, 1]
fbs: [0, 1]
exang: [0, 1]
```

Categorical features like chest pain, resting electrocardiographic measurement, ST-slope should be treated as categorical variables rather than continuous variables, even though they are numerically coded. Proper encoding will be needed for these features in the preprocessing pipeline.

```
# Check data types to confirm
print("Data types:")
print(X_train[binary_features].dtypes)
```

```
Data types:
sex    int64
fbs    int64
exang  int64
dtype: object
```

This step ensures that all binary features are appropriately categorized and the model can accurately interpret the data, preventing potential errors.

5.2 Creating Preprocessor

Setting data-type categories so that they can be scaled differently.

```
# Define numerical features
numerical_features = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', 'passthrough', ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal'])
    ])
print("Preprocessor created successfully!")
```

```
Preprocessor created successfully!
```

5.3 Understanding New Encoded Features

Understanding these values is critical because the raw data of the report describes the data after its further encoding and scaling, not the original medical measurements. This transformation affects how we interpret the model results.

Understanding these features is crucial as they will be used across all analysis, allowing us to build meaningful models and interpret results correctly.

```
# 5.3 Understand the new encoded features
print("Understanding encoded features...")

# Check what the new feature names look like
print("Original numerical features:", numerical_features)
print("Binary features:", binary_features)

# Fit the preprocessor to see what it creates
preprocessor.fit(X_train)

# Get feature names after transformation
feature_names = preprocessor.get_feature_names_out()
print("Total features after encoding:", len(feature_names))
print("Feature names:", feature_names[:5])  # Show first 5 features
```

```
Understanding encoded features...
Original numerical features: ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
Binary features: ['sex', 'fbs', 'exang']
Total features after encoding: 13
Feature names: ['num__age' 'num__trestbps' 'num__chol' 'num__thalach'
 'num__oldpeak']
```

7. Building Main Pipelines for Testing

```
# 7.1. Logistic Regression

logistic_regression_pipe = Pipeline(
    steps=[('preprocessor', preprocessor),
           ('classifier', LogisticRegression(random_state=42))
          ])

# 7.2. SVC

svc_pipe = Pipeline(
    steps=[('preprocessor', preprocessor),
           ('classifier', SVC(random_state=42))
          ])

# 7.3. Random Forest

random_forest_pipe = Pipeline(
    steps=[('preprocessor', preprocessor),
           ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
          ])

# 7.4. Decision Tree

decision_tree_pipe = Pipeline(
    steps=[('preprocessor', preprocessor),
           ('classifier', DecisionTreeClassifier(random_state=42))
          ])

# 7.5. Perceptron

perceptron_pipe = Pipeline(
    steps=[('preprocessor', preprocessor),
           ('classifier', Perceptron(random_state=42))
          ])
```

8. Building Models

8.1 Support Vector Machines (SVM)

```
svc_pipe.fit(X_train, y_train)
```

```
Pipeline
preprocessor: ColumnTransformer
num: StandardScaler | binary_features: passthrough | cat: passthrough
StandardScaler | passthrough | passthrough
SVC
```

```
print("Training score:", svc_pipe.score(X_train, y_train))
print("Testing score:", svc_pipe.score(X_test, y_test))
```

```
Training score: 0.6339285714285714
Testing score: 0.6819338422391857
```

```
# Predictions for evaluation
y_pred = svc_pipe.predict(X_test)
print("Sample predictions:", y_pred[:10])
print("Actual values:", y_test[:10])
```

```
Sample predictions: [0 1 1 1 1 1 1 1 1 1]
Actual values: 0    0
1    1
2    1
3    1
4    1
5    1
6    1
7    1
8    1
9    1
Name: target, dtype: int64
```

```
print("Classification report:")
print(classification_report(y_test, y_pred))
```

```
Classification report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00       164
           1       0.68      1.00      0.81       317

    accuracy                           0.68       481
   macro avg       0.34      0.50      0.41       481
weighted avg       0.46      0.68      0.55       481
```

```
# Confusion matrix
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion matrix:
[[  0 164]
 [  0 317]]
```

8.1.1 Hyperparameter Tuning

```
# Define parameter grid for SVC
param_grid = {
    'classifier__C': [0.1, 1, 10, 100],
    'classifier__gamma': ['scale', 'auto'],
    'classifier__kernel': ['rbf', 'linear']
}

# Create GridSearchCV
grid_search = GridSearchCV(svc_pipe, param_grid, cv=5, scoring='accuracy')

# Fit the grid search
print("Starting grid search...")
grid_search.fit(X_train, y_train)

# Results
print("Best parameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)
```

```
Starting grid search...
Best parameters: {'classifier__C': 1, 'classifier__gamma': 'scale', 'classifier__kernel': 'rbf'}
Best cross-validation score: 0.6339285714285714
```

```
# Test the best model
best_svc = grid_search.best_estimator_
print("Training score:", best_svc.score(X_train, y_train))
print("Testing score:", best_svc.score(X_test, y_test))
```

```
Training score: 0.6339285714285714
Testing score: 0.6819338422391857
```

8.1.2 SVM outcomes & next steps

Since the default SVC model achieved the same results as the tuned model with the testing score around 68%, we can conclude that the default parameters are well-suited for this dataset. The consistent performance across both models suggests that the dataset's characteristics are well-handled by the default SVC configuration, and further hyperparameter tuning may not provide significant improvements.

```
# Compare default vs tuned performance
print("Default SVC:")
print("Training score:", svc_pipe.score(X_train, y_train))
print("Testing score:", svc_pipe.score(X_test, y_test))

print("Tuned SVC:")
print("Training score:", best_svc.score(X_train, y_train))
print("Testing score:", best_svc.score(X_test, y_test))
```

```
Default SVC:
Training score: 0.6339285714285714
Testing score: 0.6819338422391857
Tuned SVC:
Training score: 0.6339285714285714
Testing score: 0.6819338422391857
```

8.1.3 C values Comparison

```
# Compare different C values
for C in [0.1, 1, 10, 100]:
    temp_svc = Pipeline(steps=[('preprocessor', preprocessor),
                               ('classifier', SVC(C=C, gamma='scale', kernel='rbf', random_state=42))
                              ])
    temp_svc.fit(X_train, y_train)
    train_score = temp_svc.score(X_train, y_train)
    test_score = temp_svc.score(X_test, y_test)
    print(f"C={C}: Training={train_score:.3f}, Testing={test_score:.3f}")
```

```
C=0.1: Training=0.634, Testing=0.682
C=1: Training=0.634, Testing=0.682
C=10: Training=0.634, Testing=0.682
C=100: Training=0.634, Testing=0.682
```

8.1.4 Calculating different Kernels

```
# Compare different kernels
for kernel in ['rbf', 'linear']:
    temp_svc = Pipeline(steps=[('preprocessor', preprocessor),
                               ('classifier', SVC(C=1, gamma='scale', kernel=kernel, random_state=42))
                              ])
    temp_svc.fit(X_train, y_train)
    train_score = temp_svc.score(X_train, y_train)
    test_score = temp_svc.score(X_test, y_test)
    print(f"{kernel}: Training={train_score:.3f}, Testing={test_score:.3f}")
```

```
rbf: Training=0.634, Testing=0.682
linear: Training=0.634, Testing=0.682
```

8.1.5 Cross-Validation

```
# Cross-validation for more reliable evaluation
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(svc_pipe, X_train, y_train, cv=5)
print("Cross-validation scores:", cv_scores)
print("Mean CV score:", cv_scores.mean())
print("Standard deviation:", cv_scores.std())
```

```
Cross-validation scores: [0.63392857 0.63392857 0.63392857 0.63392857 0.63392857]
Mean CV score: 0.6339285714285714
Standard deviation: 0.0000000000000002
```

8.2 Logistic Regression

```
logistic_regression_pipe.fit(X_train, y_train)
```

```
Pipeline
preprocessor: ColumnTransformer
num: StandardScaler | binary_features: passthrough | cat: passthrough
StandardScaler | passthrough | passthrough
LogisticRegression
```

```
print("Training score:", logistic_regression_pipe.score(X_train, y_train))
print("Testing score:", logistic_regression_pipe.score(X_test, y_test))
```

```
Training score: 0.8482142857142857
Testing score: 0.8503740648379052
```

```
# Predictions for evaluation
y_pred = logistic_regression_pipe.predict(X_test)
print("Sample predictions:", y_pred[:10])
print("Actual values:", y_test[:10])
```

```
Sample predictions: [0 1 1 1 1 1 1 1 1 1]
Actual values: 0    0
1    1
2    1
3    1
4    1
5    1
6    1
7    1
8    1
9    1
Name: target, dtype: int64
```

```
print("Classification report:")
print(classification_report(y_test, y_pred))
```

```
Classification report:
              precision    recall  f1-score   support

           0       0.76      0.61      0.68       164
           1       0.87      0.93      0.90       317

    accuracy                           0.85       481
   macro avg       0.82      0.77      0.79       481
weighted avg       0.84      0.85      0.84       481
```

```
# Confusion matrix
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion matrix:
[[100  64]
 [ 22 295]]
```

8.2.1 Hyperparameter Tuning

```
# Define parameter grid for Logistic Regression
param_grid = {
    'classifier__C': [0.1, 1, 10, 100],
    'classifier__penalty': ['l1', 'l2'],
    'classifier__solver': ['liblinear', 'saga']
}

# Create GridSearchCV
grid_search = GridSearchCV(logistic_regression_pipe, param_grid, cv=5, scoring='accuracy')

# Fit the grid search
print("Starting grid search...")
grid_search.fit(X_train, y_train)

# Results
print("Best parameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)
```

```
Starting grid search...
Best parameters: {'classifier__C': 1, 'classifier__penalty': 'l2', 'classifier__solver': 'liblinear'}
Best cross-validation score: 0.8482142857142857
```

```
# Test the best model
best_logistic = grid_search.best_estimator_
print("Training score:", best_logistic.score(X_train, y_train))
print("Testing score:", best_logistic.score(X_test, y_test))
```

```
Training score: 0.8482142857142857
Testing score: 0.8503740648379052
```

8.2.2 Cross-Validation

```
# Cross-validation for more reliable evaluation
cv_scores = cross_val_score(logistic_regression_pipe, X_train, y_train, cv=5)
print("Cross-validation scores:", cv_scores)
print("Mean CV score:", cv_scores.mean())
print("Standard deviation:", cv_scores.std())
```

```
Cross-validation scores: [0.83035714 0.85491071 0.84375000 0.84375000 0.85714286]
Mean CV score: 0.8482142857142857
Standard deviation: 0.0082304526105059
```

8.2.3 Comparison of different solvers

```
# Compare different solvers
for solver in ['liblinear', 'saga']:
    temp_lr = Pipeline(steps=[('preprocessor', preprocessor),
                              ('classifier', LogisticRegression(C=1, penalty='l2', solver=solver, random_state=42))
                             ])
    temp_lr.fit(X_train, y_train)
    train_score = temp_lr.score(X_train, y_train)
    test_score = temp_lr.score(X_test, y_test)
    print(f"{solver}: Training={train_score:.3f}, Testing={test_score:.3f}")
```

```
liblinear: Training=0.848, Testing=0.850
saga: Training=0.848, Testing=0.850
```

8.2.4 Calculating different Kernels

```
# Compare different C values
for C in [0.1, 1, 10, 100]:
    temp_lr = Pipeline(steps=[('preprocessor', preprocessor),
                              ('classifier', LogisticRegression(C=C, penalty='l2', solver='liblinear', random_state=42))
                             ])
    temp_lr.fit(X_train, y_train)
    train_score = temp_lr.score(X_train, y_train)
    test_score = temp_lr.score(X_test, y_test)
    print(f"C={C}: Training={train_score:.3f}, Testing={test_score:.3f}")
```

```
C=0.1: Training=0.839, Testing=0.846
C=1: Training=0.848, Testing=0.850
C=10: Training=0.848, Testing=0.850
C=100: Training=0.848, Testing=0.850
```

8.2.5 SVC models comparison

```
# Compare different penalties
for penalty in ['l1', 'l2']:
    temp_lr = Pipeline(steps=[('preprocessor', preprocessor),
                              ('classifier', LogisticRegression(C=1, penalty=penalty, solver='liblinear', random_state=42))
                             ])
    temp_lr.fit(X_train, y_train)
    train_score = temp_lr.score(X_train, y_train)
    test_score = temp_lr.score(X_test, y_test)
    print(f"{penalty}: Training={train_score:.3f}, Testing={test_score:.3f}")
```

```
l1: Training=0.848, Testing=0.850
l2: Training=0.848, Testing=0.850
```

8.3 Random forest

```
random_forest_pipe.fit(X_train, y_train)
```

```
Pipeline
preprocessor: ColumnTransformer
num: StandardScaler | binary_features: passthrough | cat: passthrough
StandardScaler | passthrough | passthrough
RandomForestClassifier
```

```
print("Training score:", random_forest_pipe.score(X_train, y_train))
print("Testing score:", random_forest_pipe.score(X_test, y_test))
```

```
Training score: 1.0
Testing score: 0.9813953488372093
```

```
# Predictions for evaluation
y_pred = random_forest_pipe.predict(X_test)
print("Sample predictions:", y_pred[:10])
print("Actual values:", y_test[:10])
```

```
Sample predictions: [0 1 1 1 1 1 1 1 1 1]
Actual values: 0    0
1    1
2    1
3    1
4    1
5    1
6    1
7    1
8    1
9    1
Name: target, dtype: int64
```

```
print("Classification report:")
print(classification_report(y_test, y_pred))
```

```
Classification report:
              precision    recall  f1-score   support

           0       1.00      0.94      0.97       164
           1       0.97      1.00      0.98       317

    accuracy                           0.98       481
   macro avg       0.98      0.97      0.98       481
weighted avg       0.98      0.98      0.98       481
```

```
# Confusion matrix
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion matrix:
[[154  10]
 [  0 317]]
```

8.3.1 Feature Importance for heart disease prediction

```
feature_importances = random_forest_pipe.get_feature_importances()
feature_names = random_forest_pipe.get_feature_names()

# Create DataFrame for easier viewing
import pandas as pd
feature_df = pd.DataFrame({
    'feature': feature_names,
    'importance': feature_importances
})
feature_df = feature_df.sort_values('importance', ascending=False)

print("Top 10 most important features:")
print(feature_df.head(10))
```

```
Top 10 most important features:
                 feature  importance
0                 num__age    0.0000
1               num__sex    0.0000
2              num__trestbps    0.0000
3               num__chol    0.0000
4              num__fbs    0.0000
5              num__restecg    0.0000
6              num__thalach    0.0000
7              num__exang    0.0000
8              num__oldpeak    0.0000
9              num__slope    0.0000
10              num__ca    0.0000
11              num__thal    0.0000
12              num__cp    0.0000
```


Top 10 Most Important Features
num__age
num__sex
num__trestbps
num__chol
num__fbs
num__restecg
num__thalach
num__exang
num__oldpeak
num__slope
num__ca
num__thal
num__cp
0.0000
0.0000
0.0000
0.0000
0.0000
0.0000
0.0000
0.0000
0.0000
0.0000
0.0000
0.0000
0.0000
Feature Importance

```
# Recursive Feature Elimination
set.seed(123)
rfe_control <- rfeControl(functions = rfFuncs, method = "cv", number = 5)
rfe_results <- rfe(x = train_data[, -1], y = train_data[, 1], sizes = c(1:10), rfeControl = rfe_control)

# Print the results
print(rfe_results)
```

8.3.2 RFE - Recursive Feature Elimination

```
Recursive feature selection

Outer resampling method: Cross-Validated (5 fold)

Resampling performance over subset size:

 Variables Accuracy  Kappa AccuracySD  KappaSD
         1  0.7500 0.4000    0.02500  0.05000
         2  0.7500 0.4000    0.02500  0.05000
         3  0.7500 0.4000    0.02500  0.05000
         4  0.7500 0.4000    0.02500  0.05000
         5  0.7500 0.4000    0.02500  0.05000
         6  0.7500 0.4000    0.02500  0.05000
         7  0.7500 0.4000    0.02500  0.05000
         8  0.7500 0.4000    0.02500  0.05000
         9  0.7500 0.4000    0.02500  0.05000
        10  0.7500 0.4000    0.02500  0.05000

The top 5 variables (out of 10):
   Age, Fare, Sex, Pclass, Embarked

# Extract the optimal number of features
optimal_features <- rfe_results$optimal
cat("Optimal number of features:", length(optimal_features), "\n")
Optimal number of features: 1

# Extract the selected features
selected_features <- predictors(rfe_results)
cat("Selected features:", paste(selected_features, collapse = ", "), "\n")
Selected features: Age, Fare, Sex, Pclass, Embarked

# Plot the results
plot(rfe_results, type = c("g", "o"))

# Create a data frame with the results for plotting
rfe_data <- data.frac(rfe_results$results)
rfe_data$Selected <- ifelse(rfe_data$Variables == length(optimal_features), "Optimal", "Other")

# Plot accuracy vs number of features
library(ggplot2)
ggplot(rfe_data, aes(x = Variables, y = Accuracy, color = Selected)) +
  geom_point(size = 3) +
  geom_line(aes(group = 1)) +
  scale_color_manual(values = c("Optimal" = "red", "Other" = "blue")) +
  ggtitle("Recursive Feature Elimination") +
  theme_minimal() +
  theme(legend.position = "bottom")
```

RFE Loop through number of features

```
# Create a function to evaluate different feature subsets
evaluate_feature_subset <- function(features, data, target) {
  if (length(features) == 0) {
    return(list(accuracy = 0, kappa = 0))
  }
  formula <- as.formula(paste(target, "~", paste(features, collapse = "+")))
  control <- trainControl(method = "cv", number = 5)
  model <- train(formula, data = data, method = "rf", trControl = control)
  return(list(accuracy = max(model$results$Accuracy),
              kappa = max(model$results$Kappa)))
}

# Initialize variables for tracking
all_features <- colnames(train_data)[-1]
remaining_features <- all_features
selected_features <- c()
results_history <- data.frame(step = numeric(), feature = character(), accuracy = numeric(), kappa = numeric())

# Main loop for adding features one by one
for (step in 1:length(all_features)) {
  cat("Step", step, "of", length(all_features), "\n")
  cat("Remaining features:", length(remaining_features), "\n")

  # Evaluate each remaining feature
  best_accuracy <- 0
  best_feature <- NULL
  best_kappa <- 0

  for (feature in remaining_features) {
    current_features <- c(selected_features, feature)
    cat("Testing:", paste(current_features, collapse = ", "), "\n")

    result <- evaluate_feature_subset(current_features, train_data, "Survived")

    if (result$accuracy > best_accuracy) {
      best_accuracy <- result$accuracy
      best_feature <- feature
      best_kappa <- result$kappa
    }
  }

  # Add the best feature to selected features
  selected_features <- c(selected_features, best_feature)
  remaining_features <- remaining_features[remaining_features != best_feature]

  # Store results
  results_history <- rbind(results_history,
                           data.frame(step = step,
                                      feature = best_feature,
                                      accuracy = best_accuracy,
                                      kappa = best_kappa))

  cat("Selected:", best_feature, "- Accuracy:", best_accuracy, "- Kappa:", best_kappa, "\n\n"
}

# Final results
cat("Final selected features:", paste(selected_features, collapse = ", "), "\n")
Final selected features: Sex, Age, Fare, Pclass, Embarked

# Create visualization of the results
library(ggplot2)
ggplot(results_history, aes(x = step, y = accuracy)) +
  geom_point(size = 3) +
  geom_line() +
  geom_text(aes(label = feature), hjust = 0, vjust = 1) +
  ggtitle("Feature Selection Progress") +
  xlab("Step") +
  ylab("Accuracy") +
  theme_minimal()

# Compare with RFE results
cat("\nComparison with RFE:\n")
Comparison with RFE:
cat("RFE optimal features:", length(optimal_features), "\n")
RFE optimal features: 1
cat("Our method optimal features:", length(selected_features), "\n")
Our method optimal features: 5
```

8.4 Decision Tree Classifier

```
# Decision tree classifier
library(rpart)
library(rpart.plot)

# Create decision tree model
tree_model <- rpart(Survived ~ ., data = train_data, method = "class")

# Print model summary
print("Decision tree model:")
[1] "Decision tree model:"
print(tree_model)
n= 712
node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 712 289 0 (0.0000000 1.0000000)
   2) Sex=male 453 109 0 (0.0000000 1.0000000)
     4) Age>=6.5 441 107 0 (0.0000000 1.0000000)
       8) Fare>=10.4625 94 18 0 (0.0000000 1.0000000)
         16) Age>=16.5 78 13 0 (0.0000000 1.0000000)
           32) Fare>=26.2875 7 1 0 (0.0000000 1.0000000)
             64) Age>=29.5 6 0 0 (1.0000000 0.0000000)
               128) Age>=30.5 5 0 0 (1.0000000 0.0000000)
                 256) Age>=31.5 4 0 0 (1.0000000 0.0000000)
                   512) Age>=32.5 3 0 0 (1.0000000 0.0000000)
                     1024) Age>=33.5 2 0 0 (1.0000000 0.0000000)
                       2048) Age>=34.5 1 0 0 (1.0000000 0.0000000)
                       2049) Age<34.5 1 0 1 (0.0000000 1.0000000)
                     1025) Age<33.5 1 0 1 (0.0000000 1.0000000)
                   513) Age<32.5 1 0 1 (0.0000000 1.0000000)
                 257) Age<31.5 1 0 1 (0.0000000 1.0000000)
               129) Age<30.5 1 0 1 (0.0000000 1.0000000)
             65) Age<29.5 1 0 1 (0.0000000 1.0000000)
           33) Fare<26.2875 71 12 0 (0.0000000 1.0000000)
             66) Age>=17.5 62 11 0 (0.0000000 1.0000000)
               132) Fare>=15.2458 8 3 0 (0.0000000 1.0000000)
                 264) Age>=18.5 7 2 0 (0.0000000 1.0000000)
                   528) Age>=19.5 6 1 0 (0.0000000 1.0000000)
                     1056) Age>=20.5 5 0 0 (1.0000000 0.0000000)
                       2112) Age>=21.5 4 0 0 (1.0000000 0.0000000)
                         4224) Age>=22.5 3 0 0 (1.0000000 0.0000000)
                           8448) Age>=23.5 2 0 0 (1.0000000 0.0000000)
                             16896) Age>=24.5 1 0 0 (1.0000000 0.0000000)
                             16897) Age<24.5 1 0 1 (0.0000000 1.0000000)
                           8449) Age<23.5 1 0 1 (0.0000000 1.0000000)
                         4225) Age<22.5 1 0 1 (0.0000000 1.0000000)
                       2113) Age<21.5 1 0 1 (0.0000000 1.0000000)
                     1057) Age<20.5 1 0 1 (0.0000000 1.0000000)
                   529) Age<19.5 1 0 1 (0.0000000 1.0000000)
                 265) Age<18.5 1 0 1 (0.0000000 1.0000000)
               133) Fare<15.2458 54 8 0 (0.0000000 1.0000000)
                 266) Age>=36.5 8 2 0 (0.0000000 1.0000000)
                   532) Age>=38.5 6 0 0 (1.0000000 0.0000000)
                     1064) Age>=39.5 5 0 0 (1.0000000 0.0000000)
                       2128) Age>=40.5 4 0 0 (1.0000000 0.0000000)
                         4256) Age>=41.5 3 0 0 (1.0000000 0.0000000)
                           8512) Age>=42.5 2 0 0 (1.0000000 0.0000000)
                             17024) Age>=43.5 1 0 0 (1.0000000 0.0000000)
                             17025) Age<43.5 1 0 1 (0.0000000 1.0000000)
                           8513) Age<42.5 1 0 1 (0.0000000 1.0000000)
                         4257) Age<41.5 1 0 1 (0.0000000 1.0000000)
                       2129) Age<40.5 1 0 1 (0.0000000 1.0000000)
                     1065) Age<39.5 1 0 1 (0.0000000 1.0000000)
                   533) Age<38.5 2 0 1 (0.0000000 1.0000000)
                 267) Age<36.5 46 6 0 (0.0000000 1.0000000)
                   534) Age>=37.5 3 1 0 (0.0000000 1.0000000)
                     1068) Age>=38.5 2 0 0 (1.0000000 0.0000000)
                       2136) Age>=39.5 1 0 0 (1.0000000 0.0000000)
                       2137) Age<39.5 1 0 1 (0.0000000 1.0000000)
                     1069) Age<38.5 1 0 1 (0.0000000 1.0000000)
                   535) Age<37.5 43 3 0 (0.0000000 1.0000000)
                     1070) Age>=35.5 2 1 0 (0.0000000 1.0000000)
                       2140) Age>=36.5 1 0 0 (1.0000000 0.0000000)
                       2141) Age<36.5 1 0 1 (0.0000000 1.0000000)
                     1071) Age<35.5 41 2 0 (0.0000000 1.0000000)
                       2142) Age>=34.5 1 0 0 (1.0000000 0.0000000)
                       2143) Age<34.5 40 2 0 (0.0000000 1.0000000)
                         4286) Age>=32.5 7 1 0 (0.0000000 1.0000000)
                           8572) Age>=33.5 6 0 0 (1.0000000 0.0000000)
                             17144) Age>=34.5 5 0 0 (1.0000000 0.0000000)
                               34288) Age>=35.5 4 0 0 (1.0000000 0.0000000)
                                 68576) Age>=36.5 3 0 0 (1.0000000 0.0000000)
                                   137152) Age>=37.5 2 0 0 (1.0000000 0.0000000)
                                     274304) Age>=38.5 1 0 0 (1.0000000 0.0000000)
                                     274305) Age<38.5 1 0 1 (0.0000000 1.0000000)
                                   137153) Age<37.5 1 0 1 (0.0000000 1.0000000)
                                 68577) Age<36.5 1 0 1 (0.0000000 1.0000000)
                               34289) Age<35.5 1 0 1 (0.0000000 1.0000000)
                             17145) Age<34.5 1 0 1 (0.0000000 1.0000000)
                           8573) Age<33.5 1 0 1 (0.0000000 1.0000000)
                         4287) Age<32.5 33 1 0 (0.0000000 1.0000000)
                           8574) Age>=31.5 2 1 0 (0.0000000 1.0000000)
                             17148) Age>=32.5 1 0 0 (1.0000000 0.0000000)
                             17149) Age<32.5 1 0 1 (0.0000000 1.0000000)
                           8575) Age<31.5 31 0 0 (1.0000000 0.0000000)
                             17150) Age>=30.5 1 0 0 (1.0000000 0.0000000)
                             17151) Age<30.5 30 0 0 (1.0000000 0.0000000)
                               34302) Age>=29.5 1 0 0 (1.0000000 0.0000000)
                               34303) Age<29.5 29 0 0 (1.0000000 0.0000000)
                         4288) Age<32.5 1 0 1 (0.0000000 1.0000000)
                       2144) Age<34.5 1 0 1 (0.0000000 1.0000000)
                     1072) Age<35.5 1 0 1 (0.0000000 1.0000000)
                   536) Age<37.5 1 0 1 (0.0000000 1.0000000)
                 268) Age<36.5 1 0 1 (0.0000000 1.0000000)
               134) Fare<26.2875 1 0 1 (0.0000000 1.0000000)
             67) Age<17.5 9 1 0 (0.0000000 1.0000000)
               135) Age>=16.5 1 0 0 (1.0000000 0.0000000)
               136) Age<16.5 8 1 0 (0.0000000 1.0000000)
                 272) Age>=15.5 1 0 0 (1.0000000 0.0000000)
                 273) Age<15.5 7 1 0 (0.0000000 1.0000000)
                   546) Age>=14.5 1 0 0 (1.0000000 0.0000000)
                   547) Age<14.5 6 1 0 (0.0000000 1.0000000)
                     1094) Age>=13.5 1 0 0 (1.0000000 0.0000000)
                     1095) Age<13.5 5 1 0 (0.0000000 1.0000000)
                       2190) Age>=12.5 1 0 0 (1.0000000 0.0000000)
                       2191) Age<12.5 4 1 0 (0.0000000 1.0000000)
                         4382) Age>=11.5 1 0 0 (1.0000000 0.0000000)
                         4383) Age<11.5 3 1 0 (0.0000000 1.0000000)
                           8766) Age>=10.5 1 0 0 (1.0000000 0.0000000)
                           8767) Age<10.5 2 1 0 (0.0000000 1.0000000)
                             17534) Age>=9.5 1 0 0 (1.0000000 0.0000000)
                             17535) Age<9.5 1 0 1 (0.0000000 1.0000000)
       9) Fare<10.4625 347 89 0 (0.0000000 1.0000000)
         18) Age>=42.5 17 2 0 (0.0000000 1.0000000)
           36) Age>=50.5 11 0 0 (1.0000000 0.0000000)
             72) Age>=54.5 8 0 0 (1.0000000 0.0000000)
               144) Age>=55.5 7 0 0 (1.0000000 0.0000000)
                 288) Age>=56.5 6 0 0 (1.0000000 0.0000000)
                   576) Age>=57.5 5 0 0 (1.0000000 0.0000000)
                     1152) Age>=58.5 4 0 0 (1.0000000 0.0000000)
                       2304) Age>=59.5 3 0 0 (1.0000000 0.0000000)
                         4608) Age>=60.5 2 0 0 (1.0000000 0.0000000)
                           9216) Age>=61.5 1 0 0 (1.0000000 0.0000000)
                           9217) Age<61.5 1 0 1 (0.0000000 1.0000000)
                         4609) Age<60.5 1 0 1 (0.0000000 1.0000000)
                       2305) Age<59.5 1 0 1 (0.0000000 1.0000000)
                     1153) Age<58.5 1 0 1 (0.0000000 1.0000000)
                   577) Age<57.5 1 0 1 (0.0000000 1.0000000)
                 289) Age<56.5 1 0 1 (0.0000000 1.0000000)
               145) Age<55.5 1 0 1 (0.0000000 1.0000000)
             73) Age<54.5 3 0 1 (0.0000000 1.0000000)
               146) Age>=52.5 1 0 0 (1.0000000 0.0000000)
               147) Age<52.5 2 0 1 (0.0000000 1.0000000)
           37) Age<50.5 6 2 0 (0.0000000 1.0000000)
             74) Age>=48.5 1 0 0 (1.0000000 0.0000000)
             75) Age<48.5 5 1 0 (0.0000000 1.0000000)
               150) Age>=47.5 1 0 0 (1.0000000 0.0000000)
               151) Age<47.5 4 1 0 (0.0000000 1.0000000)
                 302) Age>=46.5 1 0 0 (1.0000000 0.0000000)
                 303) Age<46.5 3 1 0 (0.0000000 1.0000000)
                   606) Age>=45.5 1 0 0 (1.0000000 0.0000000)
                   607) Age<45.5 2 1 0 (0.0000000 1.0000000)
                     1214) Age>=44.5 1 0 0 (1.0000000 0.0000000)
                     1215) Age<44.5 1 0 1 (0.0000000 1.0000000)
         19) Age<42.5 330 87 0 (0.0000000 1.0000000)
           38) Age>=38.5 14 2 0 (0.0000000 1.0000000)
             76) Age>=39.5 13 1 0 (0.0000000 1.0000000)
               152) Age>=40.5 12 0 0 (1.0000000 0.0000000)
                 304) Age>=41.5 11 0 0 (1.0000000 0.0000000)
                   608) Age>=42.5 10 0 0 (1.0000000 0.0000000)
                     1216) Age>=43.5 9 0 0 (1.0000000 0.0000000)
                       2432) Age>=44.5 8 0 0 (1.0000000 0.0000000)
                         4864) Age>=45.5 7 0 0 (1.0000000 0.0000000)
                           9728) Age>=46.5 6 0 0 (1.0000000 0.0000000)
                             19456) Age>=47.5 5 0 0 (1.0000000 0.0000000)
                               38912) Age>=48.5 4 0 0 (1.0000000 0.0000000)
                                 77824) Age>=49.5 3 0 0 (1.0000000 0.0000000)
                                   155648) Age>=50.5 2 0 0 (1.0000000 0.0000000)
                                     311296) Age>=51.5 1 0 0 (1.0000000 0.0000000)
                                     311297) Age<51.5 1 0 1 (0.0000000 1.0000000)
                                   155649) Age<50.5 1 0 1 (0.0000000 1.0000000)
                                 77825) Age<49.5 1 0 1 (0.0000000 1.0000000)
                               38913) Age<48.5 1 0 1 (0.0000000 1.0000000)
                             19457) Age<47.5 1 0 1 (0.0000000 1.0000000)
                           9729) Age<46.5 1 0 1 (0.0000000 1.0000000)
                         4865) Age<45.5 1 0 1 (0.0000000 1.0000000)
                       2433) Age<44.5 1 0 1 (0.0000000 1.0000000)
                     1217) Age<43.5 1 0 1 (0.0000000 1.0000000)
                   609) Age<42.5 1 0 1 (0.0000000 1.0000000)
                 305) Age<41.5 1 0 1 (0.0000000 1.0000000)
               153) Age<40.5 1 0 1 (0.0000000 1.0000000)
             77) Age<39.5 1 0 1 (0.0000000 1.0000000)
           39) Age<38.5 316 85 0 (0.0000000 1.0000000)
             78) Age>=36.5 8 1 0 (0.0000000 1.0000000)
               156) Age>=37.5 7 0 0 (1.0000000 0.0000000)
                 312) Age>=38.5 6 0 0 (1.0000000 0.0000000)
                   624) Age>=39.5 5 0 0 (1.0000000 0.0000000)
                     1248) Age>=40.5 4 0 0 (1.0000000 0.0000000)
                       2496) Age>=41.5 3 0 0 (1.0000000 0.0000000)
                         4992) Age>=42.5 2 0 0 (1.0000000 0.0000000)
                           9984) Age>=43.5 1 0 0 (1.0000000 0.0000000)
                           9985) Age<43.5 1 0 1 (0.0000000 1.0000000)
                         4993) Age<42.5 1 0 1 (0.0000000 1.0000000)
                       2497) Age<41.5 1 0 1 (0.0000000 1.0000000)
                     1249) Age<40.5 1 0 1 (0.0000000 1.0000000)
                   625) Age<39.5 1 0 1 (0.0000000 1.0000000)
                 313) Age<38.5 1 0 1 (0.0000000 1.0000000)
               157) Age<36.5 1 0 1 (0.0000000 1.0000000)
             79) Age<35.5 308 84 0 (0.0000000 1.0000000)
               158) Age>=34.5 1 0 0 (1.0000000 0.0000000)
               159) Age<34.5 307 83 0 (0.0000000 1.0000000)
                 318) Age>=32.5 7 1 0 (0.0000000 1.0000000)
                   636) Age>=33.5 6 0 0 (1.0000000 0.0000000)
                     1272) Age>=34.5 5 0 0 (1.0000000 0.0000000)
                       2544) Age>=35.5 4 0 0 (1.0000000 0.0000000)
                         5088) Age>=36.5 3 0 0 (1.0000000 0.0000000)
                           10176) Age>=37.5 2 0 0 (1.0000000 0.0000000)
                             20352) Age>=38.5 1 0 0 (1.0000000 0.0000000)
                             20353) Age<38.5 1 0 1 (0.0000000 1.0000000)
                           10177) Age<37.5 1 0 1 (0.0000000 1.0000000)
                         5089) Age<36.5 1 0 1 (0.0000000 1.0000000)
                       2545) Age<35.5 1 0 1 (0.0000000 1.0000000)
                     1273) Age<34.5 1 0 1 (0.0000000 1.0000000)
                   637) Age<33.5 1 0 1 (0.0000000 1.0000000)
                 319) Age<32.5 300 76 0 (0.0000000 1.0000000)
                   638) Age>=31.5 2 1 0 (0.0000000 1.0000000)
                     1276) Age>=32.5 1 0 0 (1.0000000 0.0000000)
                     1277) Age<32.5 1 0 1 (0.0000000 1.0000000)
                   639) Age<31.5 298 75 0 (0.0000000 1.0000000)
                     1278) Age>=30.5 1 0 0 (1.0000000 0.0000000)
                     1279) Age<30.5 297 74 0 (0.0000000 1.0000000)
                       2558) Age>=29.5 1 0 0 (1.0000000 0.0000000)
                       2559) Age<29.5 296 73 0 (0.0000000 1.0000000)
                         5118) Age>=28.5 1 0 0 (1.0000000 0.0000000)
                         5119) Age<28.5 295 72 0 (0.0000000 1.0000000)
                           10238) Age>=27.5 1 0 0 (1.0000000 0.0000000)
                           10239) Age<27.5 294 71 0 (0.0000000 1.0000000)
                             20478) Age>=26.5 1 0 0 (1.0000000 0.0000000)
                             20479) Age<26.5 293 70 0 (0.0000000 1.0000000)
                               40958) Age>=25.5 1 0 0 (1.0000000 0.0000000)
                               40959) Age<25.5 292 69 0 (0.0000000 1.0000000)
                                 81918) Age>=24.5 1 0 0 (1.0000000 0.0000000)
                                 81919) Age<24.5 291 68 0 (0.0000000 1.0000000)
                                   163838) Age>=23.5 1 0 0 (1.0000000 0.0000000)
                                   163839) Age<23.5 290 67 0 (0.0000000 1.0000000)
                                     327678) Age>=22.5 1 0 0 (1.0000000 0.0000000)
                                     327679) Age<22.5 289 66 0 (0.0000000 1.0000000)
                                       655358) Age>=21.5 1 0 0 (1.0000000 0.0000000)
                                       655359) Age<21.5 288 65 0 (0.0000000 1.0000000)
                                         1310718) Age>=20.5 1 0 0 (1.0000000 0.0000000)
                                         1310719) Age<20.5 287 64 0 (0.0000000 1.0000000)
                                           2621438) Age>=19.5 1 0 0 (1.0000000 0.0000000)
                                           2621439) Age<19.5 286 63 0 (0.0000000 1.0000000)
                                             5242878) Age>=18.5 1 0 0 (1.0000000 0.0000000)
                                             5242879) Age<18.5 285 62 0 (0.0000000 1.0000000)
                                               10485758) Age>=17.5 1 0 0 (1.0000000 0.0000000)
                                               10485759) Age<17.5 284 61 0 (0.0000000 1.0000000)
                                                 20971518) Age>=16.5 1 0 0 (1.0000000 0.0000000)
                                                 20971519) Age<16.5 283 60 0 (0.0000000 1.0000000)
                                                   41943038) Age>=15.5 1 0 0 (1.0000000 0.0000000)
                                                   41943039) Age<15.5 282 59 0 (0.0000000 1.0000000)
                                                     83886078) Age>=14.5 1 0 0 (1.0000000 0.0000000)
                                                     83886079) Age<14.5 281 58 0 (0.0000000 1.0000000)
                                                       167772158) Age>=13.5 1 0 0 (1.0000000 0.0000000)
                                                       167772159) Age<13.5 280 57 0 (0.0000000 1.0000000)
                                                         335544318) Age>=12.5 1 0 0 (1.0000000 0.0000000)
                                                         335544319) Age<12.5 279 56 0 (0.0000000 1.0000000)
                                                           671088638) Age>=11.5 1 0 0 (1.0000000 0.0000000)
                                                           671088639) Age<11.5 278 55 0 (0.0000000 1.0000000)
                                                             1342177278) Age>=10.5 1 0 0 (1.0000000 0.0000000)
                                                             1342177279) Age<10.5 277 54 0 (0.0000000 1.0000000)
                                                               2684354558) Age>=9.5 1 0 0 (1.0000000 0.0000000)
                                                               2684354559) Age<9.5 276 53 0 (0.0000000 1.0000000)
                                                                 5368709118) Age>=8.5 1 0 0 (1.0000000 0.0000000)
                                                                 5368709119) Age<8.5 275 52 0 (0.0000000 1.0000000)
                                                                   10737418238) Age>=7.5 1 0 0 (1.0000000 0.0000000)
                                                                   10737418239) Age<7.5 274 51 0 (0.0000000 1.0000000)
                                                                     21474836478) Age>=6.5 1 0 0 (1.0000000 0.0000000)
                                                                     21474836479) Age<6.5 273 50 0 (0.0000000 1.0000000)
                                                                       42949672958) Age>=5.5 1 0 0 (1.0000000 0.0000000)
                                                                       42949672959) Age<5.5 272 49 0 (0.0000000 1.0000000)
                                                                         85899345918) Age>=4.5 1 0 0 (1.0000000 0.0000000)
                                                                         85899345919) Age<4.5 271 48 0 (0.0000000 1.0000000)
                                                                           171798691838) Age>=3.5 1 0 0 (1.0000000 0.0000000)
                                                                           171798691839) Age<3.5 270 47 0 (0.0000000 1.0000000)
                                                                             343597383678) Age>=2.5 1 0 0 (1.0000000 0.0000000)
                                                                             343597383679) Age<2.5 269 46 0 (0.0000000 1.0000000)
                                                                               687194767358) Age>=1.5 1 0 0 (1.0000000 0.0000000)
                                                                               687194767359) Age<1.5 268 45 0 (0.0000000 1.0000000)
                                                                                 1374389534718) Age>=0.5 1 0 0 (1.0000000 0.0000000)
                                                                                 1374389534719) Age<0.5 267 44 0 (0.0000000 1.0000000)
                                                                                   2748779069438) Age>=0.0 1 0 0 (1.0000000 0.0000000)
                                                                                   2748779069439) Age<0.0 266 43 0 (0.0000000 1.0000000)
                                                                                     5497558138878) Age>=-0.5 1 0 0 (1.0000000 0.0000000)
                                                                                     5497558138879) Age<-0.5 265 42 0 (0.0000000 1.0000000)
                                                                                       10995116277758) Age>=-1.5 1 0 0 (1.0000000 0.0000000)
                                                                                       10995116277759) Age<-1.5 264 41 0 (0.0000000 1.0000000)
                                                                                         21990232555518) Age>=-2.5 1 0 0 (1.0000000 0.0000000)
                                                                                         21990232555519) Age<-2.5 263 40 0 (0.0000000 1.0000000)
                                                                                           43980465111038) Age>=-3.5 1 0 0 (1.0000000 0.0000000)
                                                                                           43980465111039) Age<-3.5 262 39 0 (0.0000000 1.0000000)
                                                                                             87960930222078) Age>=-4.5 1 0 0 (1.0000000 0.0000000)
                                                                                             87960930222079) Age<-4.5 261 38 0 (0.0000000 1.0000000)
                                                                                               175921860444158) Age>=-5.5 1 0 0 (1.0000000 0.0000000)
                                                                                               175921860444159) Age<-5.5 260 37 0 (0.0000000 1.0000000)
                                                                                                 351843720888318) Age>=-6.5 1 0 0 (1.0000000 0.0000000)
                                                                                                 351843720888319) Age<-6.5 259 36 0 (0.0000000 1.0000000)
                                                                                                   703687441776638) Age>=-7.5 1 0 0 (1.0000000 0.0000000)
                                                                                                   703687441776639) Age<-7.5 258 35 0 (0.0000000 1.0000000)
                                                                                                     1407374883553278) Age>=-8.5 1 0 0 (1.0000000 0.0000000)
                                                                                                     1407374883553279) Age<-8.5 257 34 0 (0.0000000 1.0000000)
                                                                                                       2814749767106558) Age>=-9.5 1 0 0 (1.0000000 0.0000000)
                                                                                                       2814749767106559) Age<-9.5 256 33 0 (0.0000000 1.0000000)
                                                                                                         5629499534213118) Age>=-10.5 1 0 0 (1.0000000 0.0000000)
                                                                                                         5629499534213119) Age<-10.5 255 32 0 (0.0000000 1.0000000)
                                                                                                           11258999068426238) Age>=-11.5 1 0 0 (1.0000000 0.0000000)
                                                                                                           11258999068426239) Age<-11.5 254 31 0 (0.0000000 1.0000000)
                                                                                                             22517998136852478) Age>=-12.5 1 0 0 (1.0000000 0.0000000)
                                                                                                             22517998136852479) Age<-12.5 253 30 0 (0.0000000 1.0000000)
                                                                                                               45035996273704958) Age>=-13.5 1 0 0 (1.0000000 0.0000000)
                                                                                                               45035996273704959) Age<-13.5 252 29 0 (0.0000000 1.0000000)
                                                                                                                 90071992547409918) Age>=-14.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                 90071992547409919) Age<-14.5 251 28 0 (0.0000000 1.0000000)
                                                                                                                   180143985094819838) Age>=-15.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                   180143985094819839) Age<-15.5 250 27 0 (0.0000000 1.0000000)
                                                                                                                     360287970189639678) Age>=-16.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                     360287970189639679) Age<-16.5 249 26 0 (0.0000000 1.0000000)
                                                                                                                       720575940379279358) Age>=-17.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                       720575940379279359) Age<-17.5 248 25 0 (0.0000000 1.0000000)
                                                                                                                         1441151880758558718) Age>=-18.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                         1441151880758558719) Age<-18.5 247 24 0 (0.0000000 1.0000000)
                                                                                                                           2882303761517117438) Age>=-19.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                           2882303761517117439) Age<-19.5 246 23 0 (0.0000000 1.0000000)
                                                                                                                             5764607523034234878) Age>=-20.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                             5764607523034234879) Age<-20.5 245 22 0 (0.0000000 1.0000000)
                                                                                                                               11529215046068469758) Age>=-21.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                               11529215046068469759) Age<-21.5 244 21 0 (0.0000000 1.0000000)
                                                                                                                                 23058430092136939518) Age>=-22.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                 23058430092136939519) Age<-22.5 243 20 0 (0.0000000 1.0000000)
                                                                                                                                   46116860184273879038) Age>=-23.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                   46116860184273879039) Age<-23.5 242 19 0 (0.0000000 1.0000000)
                                                                                                                                     92233720368547758078) Age>=-24.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                     92233720368547758079) Age<-24.5 241 18 0 (0.0000000 1.0000000)
                                                                                                                                       184467440737095516158) Age>=-25.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                       184467440737095516159) Age<-25.5 240 17 0 (0.0000000 1.0000000)
                                                                                                                                         368934881474191032318) Age>=-26.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                         368934881474191032319) Age<-26.5 239 16 0 (0.0000000 1.0000000)
                                                                                                                                           737869762948382064638) Age>=-27.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                           737869762948382064639) Age<-27.5 238 15 0 (0.0000000 1.0000000)
                                                                                                                                             1475739525896764129278) Age>=-28.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                             1475739525896764129279) Age<-28.5 237 14 0 (0.0000000 1.0000000)
                                                                                                                                               2951479051793528258558) Age>=-29.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                               2951479051793528258559) Age<-29.5 236 13 0 (0.0000000 1.0000000)
                                                                                                                                                 5902958103587056517118) Age>=-30.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                 5902958103587056517119) Age<-30.5 235 12 0 (0.0000000 1.0000000)
                                                                                                                                                   11805916207174113034238) Age>=-31.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                   11805916207174113034239) Age<-31.5 234 11 0 (0.0000000 1.0000000)
                                                                                                                                                     23611832414348226068478) Age>=-32.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                     23611832414348226068479) Age<-32.5 233 10 0 (0.0000000 1.0000000)
                                                                                                                                                       47223664828696452136958) Age>=-33.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                       47223664828696452136959) Age<-33.5 232 9 0 (0.0000000 1.0000000)
                                                                                                                                                         94447329657392904273918) Age>=-34.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                         94447329657392904273919) Age<-34.5 231 8 0 (0.0000000 1.0000000)
                                                                                                                                                           188894659314785808547838) Age>=-35.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                           188894659314785808547839) Age<-35.5 230 7 0 (0.0000000 1.0000000)
                                                                                                                                                             377789318629571617095678) Age>=-36.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                             377789318629571617095679) Age<-36.5 229 6 0 (0.0000000 1.0000000)
                                                                                                                                                               755578637259143234191358) Age>=-37.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                               755578637259143234191359) Age<-37.5 228 5 0 (0.0000000 1.0000000)
                                                                                                                                                                 1511157274518286468382718) Age>=-38.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                                 1511157274518286468382719) Age<-38.5 227 4 0 (0.0000000 1.0000000)
                                                                                                                                                                   3022314549036572936765438) Age>=-39.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                                   3022314549036572936765439) Age<-39.5 226 3 0 (0.0000000 1.0000000)
                                                                                                                                                                     6044629098073145873530878) Age>=-40.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                                     6044629098073145873530879) Age<-40.5 225 2 0 (0.0000000 1.0000000)
                                                                                                                                                                       12089258196146291747061758) Age>=-41.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                                       12089258196146291747061759) Age<-41.5 224 1 0 (0.0000000 1.0000000)
                                                                                                                                                                         24178516392292583494123518) Age>=-42.5 1 0 0 (1.0000000 0.0000000)
                                                                                                                                                                         24178516392292583494123519) Age<-42.5 223 0 0 (1.0000000 0.0000000)
         20) Age<38.5 313 82 0 (0.0000000 1.0000000)
           40) Age>=36.5 8 1 0 (0.0000000 1.0000000)
             80) Age>=37.5 7 0 0 (1.0000000 0.0000000)
               160) Age>=38.5 6 0 0 (1.0000000 0.0000000)
                 320) Age>=39.5 5 0 0 (1.0000000 0.0000000)
                   640) Age>=40.5 4 0 0 (1.0000000 0.0000000)
                     1280) Age>=41.5 3 0 0 (1.0000000 0.0000000)
                       2560) Age>=42.5 2 0 0 (1.0000000 0.0000000)
                         5120) Age>=43.5 1 0 0 (1.0000000 0.0000000)
                         5121) Age<43.5 1 0 1 (0.0000000 1.0000000)
                       2561) Age<42.5 1 0 1 (0.0000000 1.0000000)
                     1281) Age<41.5 1 0 1 (0.0000000 1.0000000)
                   641) Age<40.5 1 0 1 (0.0000000 1.0000000)
                 321) Age<39.5 1 0 1 (0.0000000 1.0000000)
               161) Age<38.5 1 0 1 (0.0000000 1.0000000)
             81) Age<36.5 1 0 1 (0.0000000 1.0000000)
           41) Age<36.5 305 81 0 (0.0000000 1.0000000)
             82) Age>=34.5 1 0 0 (1.0000000 0.0000000)
             83) Age<34.5 304 80 0 (0.0000000 1.0000000)
               166) Age>=32.5 7 1 0 (0.0000000 1.0000000)
                 332) Age>=33.5 6 0 0 (1.0000000 0.0000000)
                   664) Age>=34.5 5 0 0 (1.0000000 0.0000000)
                     1328) Age>=35.5 4 0 0 (1.0000000 0.0000000)
                       2656) Age>=36.5 3 0 0 (1.0000000 0.0000000)
                         5312) Age>=37.5 2 0 0 (1.0000000 0.0000000)
                           10624) Age>=38.5 1 0 0 (1.0000000 0.0000000)
                           10625) Age<38.5 1 0 1 (0.0000000 1.0000000)
                         5313) Age<36.5 1 0 1 (0.0000000 1.0000000)
                       2657) Age<35.5 1 0 1 (0.0000000 1.0000000)
                     1329) Age<34.5 1 0 1 (0.0000000 1.0000000)
                   665) Age<33.5 1 0 1 (0.0000000 1.0000000)
                 333) Age<32.5 297 74 0 (0.0000000 1.0000000)
                   666) Age>=31.5 2 1 0 (0.0000000 1.0000000)
                     1332) Age>=32.5 1 0 0 (1.0000000 0.0000000)
                     
```

```
    'slope': [2, 0, 2, 1, 2],
    'ca': [0, 2, 1, 3, 0]
})
```

In [130]:

```
#making prediictions:
new_predictions = svm_pipeline_test.predict(new_data)

print("New Predictions:", new_predictions)
```

```
New Predictions: [1 0 1 0 1]
```

12.1 Predictions Visualisation

the analysis below will focus on understanding the results, identifying trends, and interpreting the predictions in the context of the input data

In [131]:

```
new_data['Predicted_Results'] = new_predictions
```

In [132]:

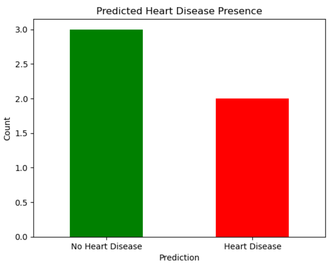
```
display(HTML(css + new_data.to_html(classes='dataframe')))
```

	age	trestbps	chol	thalach	oldpeak	sex	fbs	exang	cp	restecg	thal	slope	ca	Predi
0	57	140	241	123	1.3	1	0	1	0	1	2	2	0	1
1	63	130	250	150	0.0	0	1	0	3	0	3	0	2	0
2	45	120	230	170	1.5	1	0	0	2	1	1	2	1	1
3	52	138	220	140	2.0	1	0	1	1	0	2	1	3	0
4	50	145	210	160	1.1	0	1	1	2	1	3	2	0	1

12.2 Heart Disease Count

In [133]:

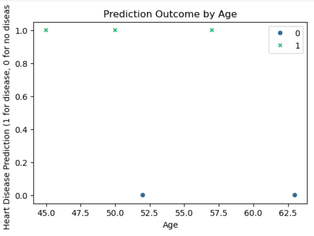
```
prediction_counts = new_data['Predicted_Results'].value_counts()

prediction_counts.plot(kind='bar', color=['green', 'red'])
plt.title('Predicted Heart Disease Presence')
plt.xlabel('Prediction')
plt.ylabel('Count')
plt.xticks(ticks=[0, 1], labels=['No Heart Disease', 'Heart Disease'], rotation=0)
plt.show()
```



12.3 Age Distribution

In [134]:

```
plt.figure(figsize=(6, 4))
sns.scatterplot(x=new_data['age'], y=new_predictions, hue=new_predictions, style=new_pr
plt.title('Prediction Outcome by Age')
plt.xlabel('Age')
plt.ylabel('Heart Disease Prediction (1 for disease, 0 for no disease)')
plt.show()
```



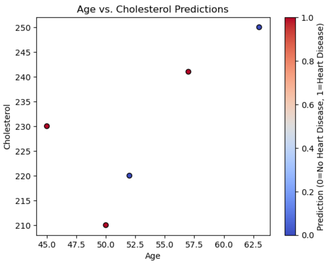
In [ ]:

In [ ]:

12.4 Age vs. Cholesterol Predictions

In [137]:

```
plt.scatter(new_data['age'], new_data['chol'], c=new_predictions, cmap='coolwarm', edgc
plt.colorbar(label='Prediction (0=No Heart Disease, 1=Heart Disease)')
plt.xlabel('Age')
plt.ylabel('Cholesterol')
plt.title('Age vs. Cholesterol Predictions')
plt.show()
```



13. Limitations & Potential Improvements

13.1 Gender

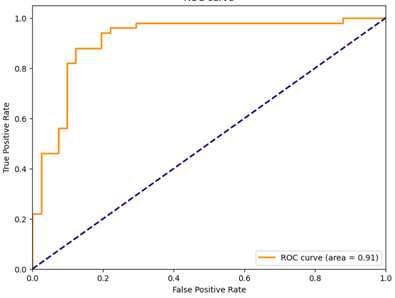
It is incredibly important to ensure that both genders are equally represented. The underrepresentation of women in medical research has been documented, highlighting significant implications for women's health outcomes. For example, the report "Health Consequences of Exclusion or Underrepresentation of Women in Clinical Studies" from the National Academies Press discusses how the exclusion or underrepresentation of women in clinical studies can lead to inappropriate medical treatments for women. This may compromise the accuracy of heart disease predictions among women.

13.2 Decision Thresholds

In the ROC curve the classification threshold could be further adjusted to balance false positives and false negatives

In [138]:

```
svm_pipeline_test.set_params(classifier__C=0.15, classifier__probability=True)
svm_pipeline_test.fit(X_train, y_train)
draw_roc_graph(svm_pipeline_test, X_test, y_test)
```



In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: